

Introduction to DBMS

Unit 1: Fundamentals of Database Management Systems

History and Evolution of Database Systems

The discipline of database management emerged during the late nineteen sixties and early nineteen seventies as organisations began to recognise the limitations of storing institutional records inside independent, application-specific files. In the earliest computing environments, every program maintained its own private files, and any two programs that needed the same information were forced to keep separate, duplicate copies of it. This arrangement is generally referred to as a traditional file-processing system, and while it was adequate for small, isolated tasks, it quickly became unmanageable as the volume of institutional data grew. The evolution from file systems to database systems was driven primarily by the need to control data redundancy, to enforce consistency across multiple applications, and to give many users concurrent access to a shared pool of information. Pioneering systems such as the Integrated Data Store and later IBM's Information Management System introduced the hierarchical and network data models, which organised records using explicit pointers and parent-child relationships. The publication of Edgar Codd's relational model in nineteen seventy marked a turning point in this evolution, because it proposed that data could be represented purely as mathematical relations, or tables, without requiring the application programmer to understand the underlying physical storage structure. This conceptual shift laid the theoretical foundation for the modern relational database management system, and it is the reason that Structured Query Language remains the dominant means of interacting with institutional data today.

Purpose, Advantages and Data Independence

A database management system, commonly abbreviated as DBMS, is software that allows users to define, create, maintain and control access to a database. Its central purpose is to act as an intermediary between the physical data stored on disk and the many application programs and users who need to query or modify that data. When compared with a traditional file-processing approach, a database management system offers several decisive advantages. First, it minimises data redundancy by allowing multiple applications to share a single, centrally maintained copy of every fact, which in turn reduces the storage space consumed and the effort required to keep duplicate copies synchronised. Second, it improves data consistency, since an update made in one place is immediately visible to every other program that reads the same table. Third, it provides robust mechanisms for enforcing integrity constraints, for restricting unauthorised access through security and authorisation subsystems, and for recovering gracefully after a hardware or software failure. A further advantage, and one of the most important theoretical contributions of the relational approach, is the property known as data independence. Data independence is the capacity to change the schema at one level of a database system without having to change the schema at the next higher level, so that an application program continues to function correctly even after the internal storage structures have been reorganised for performance reasons.

Database Users and Administration

A database environment typically serves several distinct categories of users, each of whom interacts with the system in a different way. Naive users, sometimes called end users, interact with the database indirectly through pre-written application programs and graphical interfaces, and they generally have no knowledge of the underlying schema. Application programmers write the software that naive users rely upon, embedding database queries inside a general-purpose programming language. Sophisticated users, such as business analysts and engineers, interact directly with the system using query languages in order to satisfy complex information requirements without the help of a written program. Overseeing the entire environment is the database administrator, or DBA, whose responsibilities are both technical and managerial in nature. The database administrator is responsible for defining the conceptual schema, deciding on the storage structure and access methods used at the physical level, granting authorisation for data access, specifying integrity constraints, and monitoring performance so that the system responds efficiently as the volume of stored data increases. The administrator is also responsible for coordinating backup and recovery procedures, ensuring that the organisation can restore its data after a failure with minimal loss.

Applications, Transactions and the Role of Keys

Database management systems are now embedded in almost every sector of modern life, including banking and financial services, airline and railway reservation systems, university enrolment and examination records, hospital patient management, retail inventory control, and telecommunications billing. In every one of these settings, an operation performed on the database is described as a transaction, which is a logical unit of work composed of one or more read and write operations that must be executed in its entirety or not at all. A transaction is expected to satisfy the properties of atomicity, consistency, isolation and durability, collectively known by the acronym ACID, so that the database always moves from one consistent state to another even when many transactions execute at the same time. Concurrency is the term used to describe this simultaneous execution of multiple transactions, and the database management system employs concurrency control techniques, such as locking and timestamp ordering, to prevent one transaction from interfering with another and producing an incorrect result. Underlying all of this reliability is the humble but essential concept of the primary key, an attribute or a combination of attributes that uniquely identifies every row in a table. Because a primary key can never contain a duplicate or a missing value, it becomes the anchor point through which transactions locate, lock and update exactly the record that was intended, which is why the correct choice of primary key is considered one of the most important decisions in the entire process of database design.

Prepared for classroom distribution as part of the Acadrium academic content repository. For internal educational use only.